Rockchip Linux updateEngine Upgrade Solution Introduction

ID: RK-KF-YF-348

Release version: V2.0.0

Date: 2026-06-23

Security Level: □Top-Secret □Secret □Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD. (“ROCKCHIP”) DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2026. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, No.89, Software Boulevard, TongPan Rd, Fuzhou, Fujian, PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document describes how to use the Rockchip Linux platform upgrade solution and how to perform secondary development.

Intended Audience

This document (this guide) is mainly intended for:

- Technical support engineers
- Software development engineers

Revision History

Date	Version	Author	Revision Description
2019-06-05	V1.0.1	HKH MLC	Initial version
2020-03-31	V1.0.2	ZLX	Format update
2020-12-14	V1.0.3	ZLX	Updated company name and document format
2021-02-18	V1.1.0	CWW	1. Added SD card boot upgrade support 2. Added A/B system upgrade support
2023-03-16	V1.2.0	ZSQ	Added differential OTA support
2026-02-26	V1.3.0	ZJH	Added MTD/UBI differential upgrade support
2026-06-23	V2.0.0	ZJH	Updated current updateEngine parameters, Buildroot configuration, OTA upgrade scope, GPT/IDBlock and pre-write checks, and dual-storage firmware packaging

Contents

Rockchip Linux updateEngine Upgrade Solution Introduction

1. Overview and Code
   - 1.1 Overview
   - 1.2 Code
2. Recovery Mode
   - 2.1 Overview
   - 2.2 Configuration and Build
   - 2.3 OTA Upgrade
   - 2.4 Log Checking
3. Differential Upgrade
   - 3.1 Dependencies and Build
   - 3.2 Partition Size Requirements
   - 3.3 Build Differential Package
   - 3.4 MTD/UBI Differential Upgrade
4. Linux A/B Mode
   - 4.1 Overview
   - 4.2 Boot Process and Data Format
     - 4.2.1 Data Format and Storage
     - 4.2.2 Boot Process
     - 4.2.3 Boot Process Flow Chart
   - 4.3 Build Configuration
     - 4.3.1 U-Boot
     - 4.3.2 Buildroot
     - 4.3.3 Partition Table
     - 4.3.4 Firmware Output
   - 4.4 OTA Upgrade
   - 4.5 Partition Boot Settings
     - 4.5.1 Bootable Settings
     - 4.5.2 A/B Slot Switch
5. Firmware Package Creation
   - 5.1 Common Package Types
   - 5.2 Recovery Mode OTA Package
   - 5.3 A/B OTA Package
   - 5.4 Full Firmware and SD/USB Boot Upgrade Package
   - 5.5 Dual-Storage Firmware Packaging
   - 5.6 Custom Partition Packaging
   - 5.7 Packaging Notes
6. SD/USB Boot Upgrade
   - 6.1 Create an SD Card Boot Disk
   - 6.2 Instructions for Making SD Card Boot Disk in Recovery System Mode
   - 6.3 Instructions for Making SD Card Boot Disk in A/B System Mode
     - 6.3.1 Prerequisites
     - 6.3.2 Creation Steps
7. Restore Factory Settings
8. Upgrade Program Details
   - 8.1 Parameters
     - 8.1.1 --misc
     - 8.1.2 --update
     - 8.1.3 --upgrade
     - 8.1.4 --reboot

8.1.5 --version_url / --check
8.1.6 --image_url
8.1.7 --savepath
8.1.8 parameter and bootloader Upgrade
8.1.9 Pre-Write Checks
8.2 Custom Partition Upgrade
9. Appendix
9.1 Firmware Packaging Tools
9.2 updateEngine Storage Write Implementation
9.3 Misc Partition

1. Overview and Code

1.1 Overview

The Rockchip Linux platform supports two boot schemes:

1. Recovery mode: the device has a separate recovery partition for upgrade operations.
2. Linux A/B mode: the device has two firmware slots and can switch between them.

Users can select either scheme according to product requirements.

1.2 Code

The Rockchip Linux platform uses the following upgrade solution.

Upgrade Solution	Code Path	Recovery Mode Upgrade	A/B Mode Upgrade
updateEngine	external/recovery	Supported	Supported

The current `external/recovery` version builds two programs:

Program	Install Path	Description
updateEngine	`/usr/bin/updateEngine`	Performs firmware upgrade, writes misc, switches A/B slots, restores factory settings, and is used by both normal and recovery systems.
recovery	`/usr/bin/recovery`	Recovery system entry program. It should only be installed into the recovery rootfs and should not be installed into the normal rootfs.

2. Recovery Mode

2.1 Overview

Recovery mode adds a recovery partition to the device. This partition consists of kernel, resource, and ramdisk, and is mainly used for upgrade operations. U-Boot checks fields stored in the misc partition to decide whether to boot the normal system or the recovery system. Because recovery mode runs in an independent system, it can preserve upgrade integrity when an upgrade is interrupted, for example by an unexpected power loss.

Advantages:

1. Preserves upgrade integrity.

Disadvantages:

1. Requires one additional partition that is only used for upgrade.
2. The upgrade process must reboot into recovery mode and cannot be completed directly in the normal system.

Partition Description:

Partition Name	Image Name	Description
loader	MiniLoaderAll.bin	First-stage loader
u-boot	uboot.img	Second-stage loader
trust	trust.img	Secure environment, such as OP-TEE and ATF. Some platforms merge trust and uboot.
misc	misc.img	Boot parameter partition
recovery	recovery.img	Root filesystem consisting of kernel, dtb, and ramdisk
boot	boot.img	kernel + dtb
rootfs	rootfs.img	Read-only root filesystem
oem	oem.img	Vendor preset data, read/write
userdata	userdata.img	User data, read/write

2.2 Configuration and Build

Buildroot recovery rootfs configuration:

```
BR2_PACKAGE_RECOVERY=y                  # Enable upgrade-related functions
BR2_PACKAGE_RECOVERY_USE_UPDATEENGINE=y # Use updateEngine
BR2_PACKAGE_RECOVERY_RECOVERYBIN=y      # Install recovery into recovery rootfs
BR2_PACKAGE_RECOVERY_UPDATEENGINEBIN=y  # Install updateEngine
BR2_PACKAGE_RECOVERY_NO_UI=y            # Disable UI; leave unset for products with
display
BR2_PACKAGE_RECOVERY_SOURCE_URL=y       # Enable network download upgrade
```

Buildroot normal rootfs configuration:

```
BR2_PACKAGE_RECOVERY=y                    # Enable upgrade-related functions
BR2_PACKAGE_RECOVERY_USE_UPDATEENGINE=y   # Use updateEngine
BR2_PACKAGE_RECOVERY_UPDATEENGINEBIN=y    # Install updateEngine
BR2_PACKAGE_RECOVERY_NO_UI=y              # Disable UI
BR2_PACKAGE_RECOVERY_SOURCE_URL=y         # Enable network download upgrade
# BR2_PACKAGE_RECOVERY_RECOVERYBIN is not configured; normal rootfs must not install
recovery
```

`BR2_PACKAGE_RECOVERY_NO_UI` only affects the recovery binary UI build and UI dependencies such as libdrm and libpng. When the normal rootfs only installs updateEngine, this option is mainly used to reduce unnecessary UI build dependencies. Whether recovery has UI should be decided by the recovery rootfs configuration.

Display and No-Display Products

```
BR2_PACKAGE_RECOVERY_NO_UI=y # Disable UI
```

For products that require the recovery UI, leave `BR2_PACKAGE_RECOVERY_NO_UI` unset and ensure the rootfs includes DRM, PNG, and zlib dependencies.

The SDK enables the above configuration by default. The source directory is `external/recovery/`. If the source code is modified, rebuild as follows:

```
1. cd buildroot
2. rm -rf output/rockchip_rkxxxx/build/recovery-develop/*
3. source envsetup.sh
4. Select the rootfs configuration of a platform
5. make recovery-rebuild
6. rm -rf output/rockchip_rkxxxx_recovery/build/recovery-develop/*
7. source envsetup.sh
8. Select the recovery configuration of a platform
9. make recovery-rebuild
10. cd SDK/
11. ./build.sh
12. Reflash the firmware
```

2.3 OTA Upgrade

The upgrade supports both network download and local package upgrade. The partitions to be upgraded must be explicitly specified by `--upgrade`.

Network upgrade example:

```
updateEngine --image_url=http://172.16.21.110:8080/recovery/update.img \
    --misc=update \
    --savepath=/userdata/update.img \
    --upgrade=boot,recovery,rootfs \
    --reboot &
```

Local upgrade example:

```
updateEngine --image_url=/userdata/update.img \
    --misc=update \
    --savepath=/userdata/update.img \
    --upgrade=boot,recovery,rootfs \
    --reboot &
```

To upgrade common partitions in the package except userdata:

```
updateEngine --image_url=/userdata/update.img \
    --misc=update \
    --savepath=/userdata/update.img \
    --upgrade=all,-userdata \
    --reboot &
```

Common `--upgrade` forms:

Form	Description
`--upgrade=boot,recovery,rootfs`	Upgrade only boot, recovery, and rootfs.
`--upgrade=all`	Upgrade common partitions in the package except `package-file`, `parameter`, and `bootloader`.
`--upgrade=all,-userdata`	Upgrade partitions selected by `all` but exclude userdata. `-userdata` is recommended to avoid shell history expansion of `!userdata`.
`--upgrade=all,parameter`	Upgrade all plus parameter/GPT.
`--upgrade=all,bootloader`	Upgrade all plus bootloader/IDBlock.
`--upgrade=all,parameter,bootloader`	Commonly used for SD/USB boot full-firmware upgrade.

> **Note:** Released products and mass-production OTA packages are not recommended to upgrade `parameter`/GPT and `bootloader`/IDBlock by default. Power loss, write failure, or an incorrect layout during GPT or IDBlock upgrade may cause the device to fail to boot from the storage medium. Recovery through flashing tools or Maskrom mode may be required.

Process:

1. Firmware version comparison, optional with `--version_url` or `--check`.
2. Download the firmware or use a local firmware specified by `--image_url`, and save it locally with `--savepath`.
3. In the normal system, if `--upgrade` contains `recovery`, updateEngine upgrades the recovery partition first.
4. Write the remaining upgrade parameters to misc/BCB.
5. Reboot into the recovery system with `--reboot`.

6. The recovery system reads misc/BCB and upgrades the remaining partitions.
7. After upgrade succeeds, clear misc/BCB and the upgrade package, then reboot into the normal system.

Optional parameters:

1. `--version_url`: Remote or local path. If it is not specified, version comparison is skipped.
2. `--savepath`: Firmware save path. `/userdata/update.img` or an external media path is recommended.
3. `--reboot`: After the recovery partition is upgraded, reboot into recovery mode. If it is not specified, the user must reboot manually.

Notes:

1. `--upgrade` is mandatory except for the SD/USB automatic upgrade flow. If it is not specified, updateEngine reports an error.
2. `parameter` and `bootloader` are not included by `all`; they must be explicitly specified in `--upgrade`.
3. If the package is saved in `/userdata/update.img`, do not upgrade `userdata` in the same upgrade. Otherwise, the pre-write check detects that userdata is still mounted and aborts the upgrade.
4. Invalid forms such as `--upgrade=boot.recovery` are not automatically split into boot and recovery. Use `--upgrade=boot,recovery`.

2.4 Log Checking

1. Check upgrade logs from the serial console.
2. Check persistent logs under `/userdata/recovery/`.

After upgrade, the following log files can be checked on the device:

```
cat /userdata/recovery/log
cat /userdata/recovery/last_log
```

`log` is the appended recovery log, and `last_log` is the log from the latest recovery run.

3. Differential Upgrade

The Linux SDK provides `tools/linux/Linux_Diff_Firmware/` for creating differential upgrade packages. Supported functions include:

- uboot/trust/kernel partitions
- rootfs partition with squashfs, including block devices and UBI volumes
- NAND Flash systems using the MTD driver, supporting differential upgrade of raw MTD partitions and UBI SquashFS partitions

Unsupported functions include:

- ext2/ext4 filesystems

- encrypted partitions
- A/B mode

3.1 Dependencies and Build

PC-side tools include bsdiff, md5sum, and ubireader_extract_images. Install them as follows:

```
sudo apt install bsdiff
pip install ubi_reader
```

`ubi_reader` is used to extract squashfs data from a UBI image when creating a differential package. If `rootfs.img` is in UBI image format, this tool must be installed; otherwise, differential package creation will fail.

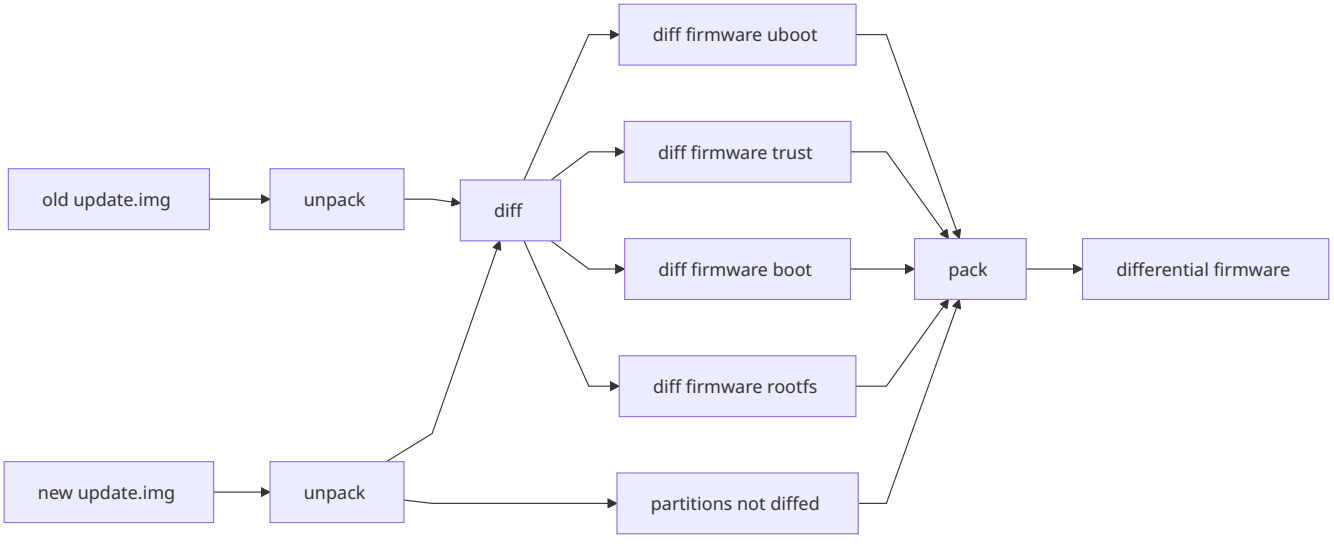
3.2 Partition Size Requirements

DDR and partition size requirements on the device side:

- DDR has no requirement and is unrelated to the differential patch package size.
- The free partition has a size requirement: it must be able to hold both `update_diff.img` and the complete new `rootfs.img` data required while applying the differential update.
    - If `update_diff.img` is stored under `/userdata/`
    - The free space in `/userdata/` should be no less than the sum of `update_diff.img` and the complete new `rootfs.img`

3.3 Build Differential Package

Create the patch package by unpacking, diffing, and packaging the old and new OTA firmware (`update.img`).



Notes

- The new `update.img` must be an OTA image, not the `update.img` used for full download flashing. It should contain only the partitions to be upgraded.
- The old firmware is recommended to use an `update.img` containing all partitions, or at least the partitions to be diffed.
- The old firmware must be identical to the firmware on the target device.

Example:

```
cd tools/linux/Linux_Diff_Firmware/
./mk-diff-ota.sh update_old.img update_new.img update_diff.img
```

The generated `update_diff.img` can be used for OTA upgrade.

3.4 MTD/UBI Differential Upgrade

For NAND Flash systems using the MTD driver, differential upgrade supports the following two partition types:

- **Raw MTD partition** (such as boot): during upgrade, old data is read from the MTD device, the bsdiff patch is applied, and the result is written back through flash_erase + nandwrite.
- **UBI SquashFS partition** (such as rootfs): during upgrade, the UBI device is attached first, old squashfs data is read from the UBI volume, the bsdiff patch is applied to generate the new squashfs, and the result is written to the UBI volume through ubiupdatevol.

When creating a differential package, if `rootfs.img` is in UBI image format, the script automatically uses `ubireader_extract_images` to extract squashfs data from the UBI image and then diffs it against the new squashfs. This ensures that the differential baseline data matches the data stored in the device-side UBI volume.

4. Linux A/B Mode

4.1 Overview

In Linux A/B mode, the device has two bootable slots. Upgrade is performed on the inactive slot while the current slot continues to run. After upgrade, the boot control data in misc is updated and the device boots from the upgraded slot.

Linux A/B has two boot slots and therefore has the following **advantages**:

1. Upgrade does not require rebooting into upgrade mode. The device can upgrade directly from the current system.
2. It prevents the device from becoming unbootable after an upgrade failure. If the new version fails to boot, the device can fall back to the currently usable version.
3. If the current system is damaged by misoperation, the system can automatically switch to the other slot.

Disadvantage:

1. Linux A/B has two slots, so it increases firmware storage usage on flash.

Partition:

Partition	Image	Description
loader	MiniLoaderAll.bin	First-stage loader, backed up 4-5 copies on the device
uboot_a	uboot.img	Second-stage loader, backed up 2 copies on the device. The backup count can be changed by modifying `u-boot/make.sh`
uboot_b	uboot.img	Backup partition of uboot_a
trust	trust.img	Security related, backed up 2 copies on the device. The backup count can be changed by modifying `u-boot/make.sh` (Some platforms merge trust and uboot into one package)
misc	misc.img	Boot parameter partition
boot_a	boot.img	kernel+dtb, boots system_a
boot_b	boot.img	kernel+dtb, boots system_b
system_a	rootfs.img	Root filesystem
system_b	rootfs.img	Root filesystem
oem	oem.img	Vendor preset partition, readable and writable
userdata	userdata.img	Data partition, readable and writable, no backup

4.2 Boot Process and Data Format

4.2.1 Data Format and Storage

A/B metadata is stored at offset 2 KB in the misc partition. The `AvbABSlotData` and `AvbABData` structures are as follows:

AvbABSlotData: stores slot_a and slot_b information.

Field	Description
`unsigned char priority`	Partition priority, 0-15. 0 means not automatically bootable, and 15 is the highest priority
`unsigned char tries_remaining`	Remaining boot attempts, maximum 7, configurable
`unsigned char successful_boot`	0: not bootable, 1: bootable
`unsigned char is_update:1`	0: upgrade failed, 1: upgrade successful. The remaining 7 bits are reserved

AvbABData: boot information for slot_a and slot_b.

Field	Description
`unsigned char magic[AVB_AB_MAGIC_LEN]`	Structure header magic: `\0AB0`
`unsigned char version_major`	Version information
`unsigned char version_minor`	Version information
`unsigned char reserved1[2]`	Reserved data
`AvbABSlotData slots[2]`	Partition boot information
`unsigned char last_boot`	Last successfully booted slot: 0 -> slot_a, 1 -> slot_b
`unsigned char reserved2[11]`	Reserved data
`unsigned char crc32`	CRC data checksum

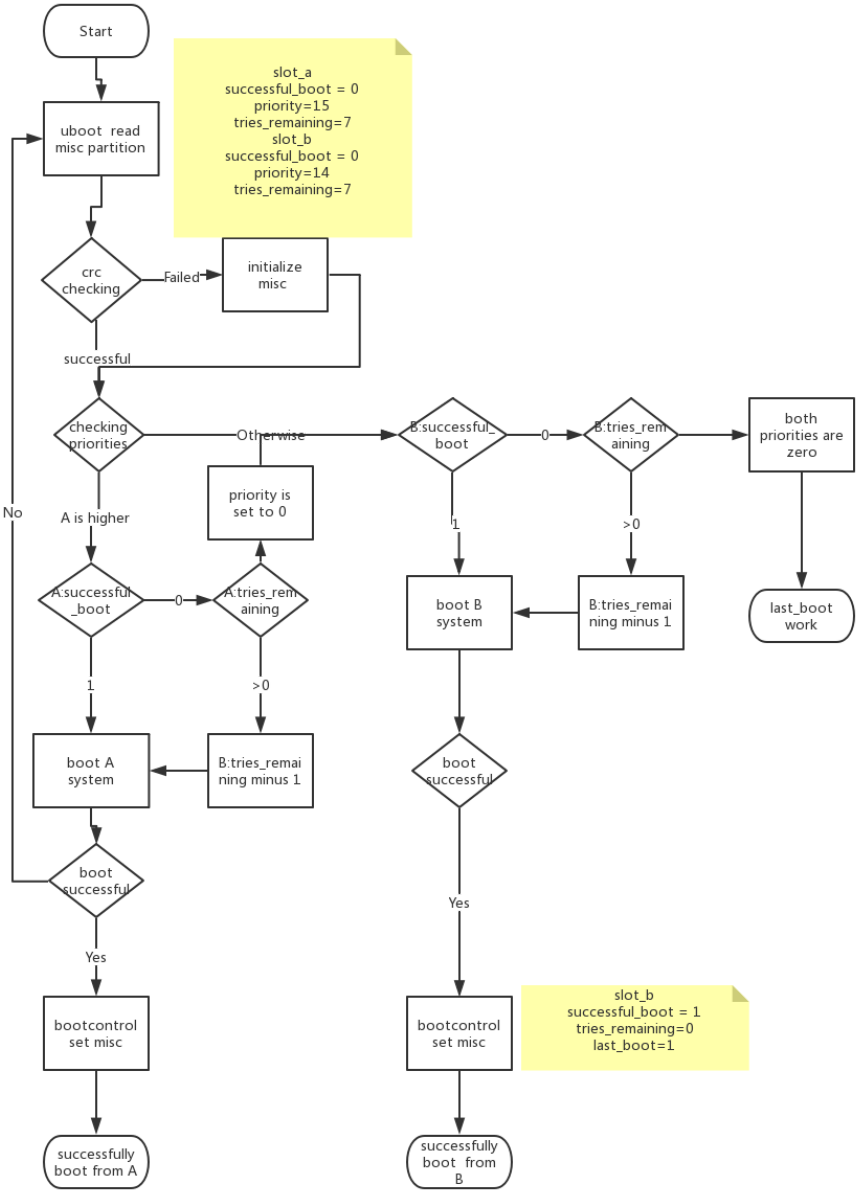
4.2.2 Boot Process

Two boot modes are supported: successful boot and reset retry.

Mode	Advantage	Disadvantage	Data set after successful boot (A boot)	Data set during upgrade (A boot, B upgrade)
Successful boot	The device does not fall back after the system boots successfully.	If storage content becomes abnormal after long operation, the system may keep rebooting.	tries_remaining=0 successful_boot=1 last_boot=0	A:priority=14 B:priority=15
Reset retry	Keeps retry mechanism to handle boot failures.	1. The device may fall back to an older version. 2. User operations may exhaust retries and cause misjudgment.	tries_remaining=7 last_boot=0	A:priority=14 B:priority=15

> **Recommendation:** NAND Flash products should prefer successful boot mode. Retry mode updates the misc partition during each boot or power-loss retry, which may increase NAND bad-block risk when performed frequently for a long time.

4.2.3 Boot Process Flow Chart



4.3 Build Configuration

4.3.1 U-Boot

Add the following configuration to defconfig, for example `u-boot/configs/rk-ab.config`:

```
CONFIG_SPL_AB=y
CONFIG_AVB_LIBAVB=y
CONFIG_AVB_LIBAVB_AB=y
CONFIG_AVB_LIBAVB_ATX=y
CONFIG_AVB_LIBAVB_USER=y
CONFIG_RK_AVB_LIBAVB_USER=y
CONFIG_ANDROID_AB=y
```

4.3.2 Buildroot

```
BR2_PACKAGE_RECOVERY=y                  # Enable upgrade function
BR2_PACKAGE_RECOVERY_BOOTCONTROL=y      # Enable boot control script
BR2_PACKAGE_RECOVERY_SUCCESSFUL_BOOT=y  # Use successful boot mode by default
BR2_PACKAGE_RECOVERY_USE_UPDATEENGINE=y # Use updateEngine
BR2_PACKAGE_RECOVERY_UPDATEENGINEBIN=y  # Compatible with old configuration
BR2_PACKAGE_RECOVERY_NO_UI=y            # Disable UI
BR2_PACKAGE_RECOVERY_SOURCE_URL=y       # HTTP network download upgrade
```

A/B normal rootfs does not need to install the `recovery` program. Do not configure `BR2_PACKAGE_RECOVERY_RECOVERYBIN`. If it is configured, the recovery program may start automatically and cause repeated reboots.

Successful boot mode is recommended by default. If the product requires reset retry, replace `BR2_PACKAGE_RECOVERY_SUCCESSFUL_BOOT=y` with `BR2_PACKAGE_RECOVERY_RETRY=y`. NAND Flash products must evaluate the lifetime and bad-block risk caused by frequent misc updates.

Rebuild after configuration:

```
1. cd buildroot
2. source envsetup.sh
3. Select the rootfs configuration of a platform
4. make recovery-rebuild
```

4.3.3 Partition Table

Example parameter partition table:

```
CMDLINE:
mtdparts=:0x00004000@0x00004000(uboot_a),0x00004000@0x00008000(uboot_b),0x00002000@0x0000c00
0(misc),0x00020000@0x0000e000(boot_a),0x00040000@0x0002e000(boot_b),0x00010000@0x0006e000(ba
ckup),0x00e00000@0x0007e000(system_a),0x00e00000@0x00e7e000(system_b),0x00040000@0x01c7e000(
oem),-@0x01cbe000(userdata:grow)
```

In A/B systems, rootfs partitions must be named `system_a/system_b`. The current U-Boot A/B logic recognizes the `system` partition name, so the device-side upgrade command uses `--upgrade=boot,system`.

Set the parameter file in `SDK/device/rockchip/rkxx`:

```
RK_AB_UPDATE=y
RK_PARAMETER="parameter-ab.txt"
```

4.3.4 Firmware Output

After selecting the board configuration, run:

```
./build.sh
```

The following firmware files are generated:

```
rockdev/
├── boot.img
├── MiniLoaderAll.bin
├── misc.img
├── oem.img
├── parameter.txt
├── rootfs.img
├── uboot.img
├── update-ab.img
├── update.img
└── userdata.img
```

> **Note:** If `RK_SDCARD_UPDATE` is configured in defconfig and `recovery.img` has been built, the SDK automatically generates `update-sd.img` during packaging. It is used to create an SD card upgrade boot disk. See Instructions for Making SD Card Boot Disk in A/B System Mode.

OTA firmware

```
./build.sh ota-updateimg
```

`update_ota.img` is generated under `rockdev` and `IMAGE`. It is used for OTA upgrade. The package usually contains `boot.img` and `rootfs.img`. Modify `./build.sh edit-ota-package-file` as required.

```
# NAME          Relative path
#
#HWDEF          HWDEF
package-file    package-file
bootloader      Image/MiniLoaderAll.bin
parameter       Image/parameter.txt
uboot_a         Image/uboot.img
boot_a          Image/boot.img
system_a        Image/rootfs.img
oem             Image/oem.img
```

Full firmware

`update-ab.img` is generated under `rockdev` and `IMAGE`. It is used for full flashing. Modify `./build.sh edit-package-file` as required.

```
# NAME          Relative path
#
#HWDEF          HWDEF
package-file    package-file
bootloader      Image/MiniLoaderAll.bin
parameter       Image/parameter.txt
misc            Image/misc.img
uboot_a         Image/uboot.img
uboot_b         Image/uboot.img
boot_a          Image/boot.img
boot_b          Image/boot.img
system_a        Image/rootfs.img
system_b        Image/rootfs.img
oem             Image/oem.img
```

4.4 OTA Upgrade

Linux A/B mode upgrades directly in the normal system and does not need to reboot into recovery. The partitions to be upgraded must be specified by `--upgrade`.

Network upgrade:

```
updateEngine --image_url=http://172.16.21.110:8080/linuxab/update_ota.img \
    --update \
    --upgrade=boot,system \
    --reboot
```

Local upgrade:

```
updateEngine --image_url=/userdata/update_ota.img \
    --update \
    --upgrade=boot,system \
    --reboot
```

Process:

1. Firmware version comparison, optional.
2. Download the firmware with `--image_url`, and save it locally with `--savepath`, optional.
3. Select the target partitions in the other slot according to the current slot. For example, if the device boots from A, upgrade `boot_b` and `system_b`.
4. Set the upgraded slot as the next boot slot.
5. Reboot.
6. Try to boot the upgraded slot.
7. After the new system boots successfully, the S99_bootcontrol script runs `updateEngine --misc=now` to mark the boot as successful.

Optional parameters:

1. `--version_url` or `--check`: if it is not specified, version comparison is skipped.
2. `--savepath`: firmware save path. If omitted, the local file specified by `--image_url` is used directly. For network download, specifying a save path is recommended.
3. `--reboot`: reboot after upgrade. If it is not specified, the user must reboot manually.

Notes:

1. In A/B systems, rootfs partitions must be named `system_a/system_b`, and the upgrade command uses `--upgrade=boot,system`.
2. Base names such as `boot` and `system` can match `boot_a/boot_b` and `system_a/system_b`.
3. A/B mode upgrades only the inactive slot. The currently booted slot is skipped.
4. `all` does not include `parameter` and `bootloader`. A/B OTA is generally not recommended to upgrade `parameter`.

4.5 Partition Boot Settings

4.5.1 Bootable Settings

Set the current slot as bootable:

```
updateEngine --misc=now
```

This command is normally executed by the `S99_bootcontrol` init script after the system starts successfully.

4.5.2 A/B Slot Switch

Switch to the other slot:

```
updateEngine --misc=other --reboot
```

Process:

1. Update A/B boot metadata at misc offset 2 KB to boot the other slot.
2. Reboot.

Optional parameter:

1. If `--reboot` is not specified, the switch takes effect on the next reboot.

Note: updateEngine sets the slot automatically after OTA upgrade; do not set it repeatedly.

5. Firmware Package Creation

A firmware package is described by `package-file`. The first column is the item name, and the second column is the image path relative to the package directory. The device-side `--upgrade` option uses the item names in the first column.Under the SDK, you can use `./build.sh edit-package-file` and `./build.sh edit-ota-package-file` to edit the package-file file.

5.1 Common Package Types

Package	Purpose	Common Build Command	Typical Contents
`update.img`	Mass-production flashing or full firmware	`./build.sh updateimg`	`parameter`, `bootloader`, `misc`, `boot`, `recovery`, `rootfs`, `userdata`, etc.
`update_ota.img`	OTA in normal or recovery system	`./build.sh ota-updateimg`	Only partitions required by this upgrade, such as `boot`, `recovery`, `rootfs`.
`update-ab.img`	A/B full firmware flashing	`./build.sh updateimg`	`boot_a`, `boot_b`, `system_a`, `system_b`, etc.
`update-sd.img`	SD boot disk for A/B products	Generated by `./build.sh updateimg` after `RK_SDCARD_UPDATE=y`	Contains recovery partition and is only used for SD boot.
`update_diff.img`	Differential OTA	`tools/linux/Linux_Diff_Firmware/mk-diff-ota.sh`	Differential patches and images not upgraded by differential patch.

5.2 Recovery Mode OTA Package

A recovery mode OTA package should only include the partitions to be upgraded.

```
# NAME          Relative path
package-file    package-file
boot            Image/boot.img
recovery        Image/recovery.img
rootfs          Image/rootfs.img
```

Device-side command:

```
updateEngine --image_url=/userdata/update_ota.img \
    --misc=update \
    --savepath=/userdata/update_ota.img \
    --upgrade=boot,recovery,rootfs \
    --reboot &
```

If the package is stored under `/userdata`, do not select userdata in the same upgrade. Otherwise the pre-write check detects that the target partition is mounted and aborts the upgrade.

5.3 A/B OTA Package

An A/B OTA package usually contains only non-shared system partitions, such as `boot_a` and `system_a`. updateEngine automatically selects the inactive slot according to the current slot.

```
# NAME          Relative path
package-file    package-file
boot_a          Image/boot.img
system_a        Image/rootfs.img
```

Device-side command:

```
updateEngine --image_url=/userdata/update_ota.img \
    --update \
    --upgrade=boot,system \
    --reboot
```

In A/B systems, rootfs partitions must be named `system_a/system_b`, so the device-side command uses `--upgrade=boot,system`.

5.4 Full Firmware and SD/USB Boot Upgrade Package

A full firmware package can contain `parameter` and `bootloader`. It is mainly used for mass-production flashing, SD/USB boot upgrade, or maintenance scenarios where partition table or IDBlock must be rewritten.

```
# NAME          Relative path
package-file    package-file
bootloader      Image/MiniLoaderAll.bin
parameter       Image/parameter.txt
misc            Image/misc.img
boot            Image/boot.img
recovery        Image/recovery.img
rootfs          Image/rootfs.img
userdata        Image/userdata.img
```

When recovery performs SD/USB boot upgrade, it automatically calls updateEngine with this default upgrade scope:

```
--upgrade=all,parameter,bootloader
```

Mass-production OTA packages are not recommended to upgrade `parameter` and `bootloader` by default. GPT and IDBlock are boot-critical data. If power loss, write failure, or layout errors occur during upgrade, the device may fail to boot from the storage medium and may require recovery by flashing tools or Maskrom mode.

5.5 Dual-Storage Firmware Packaging

A dual-storage product usually contains two types of media: boot storage and main storage. Boot storage stores boot-related firmware, such as IDBlock, U-Boot, and the `parameter.txt` for boot storage on SPI NOR. Main storage stores system partitions, such as `boot`, `rootfs`, `userdata`, and the `parameter.txt` for SATA/NVMe/eMMC.

Common combinations include NOR + SATA, NOR + NVMe, and NOR + eMMC. For storage boot principles, U-Boot configuration, and partition layout, refer to `Rockchip_Developer_Guide_Dual_Storage_EN.pdf`.

A dual-storage full firmware package should prepare image directories for each storage medium and then merge them into one `update.img`.

```
.
├── Image/                      # main storage firmware
│   ├── parameter.txt           # main storage partition table
│   ├── boot.img
│   ├── rootfs.img
│   └── userdata.img
├── Image-nor/                  # boot storage firmware
│   ├── parameter.txt           # boot storage partition table
│   ├── MiniLoaderAll.bin
│   └── uboot.img
├── afptool
└── rkImageMaker
```

Example `package-file` for main storage:

```
# NAME          Relative path
package-file    package-file
parameter       Image/parameter.txt
boot            Image/boot.img
rootfs          Image/rootfs.img
userdata        Image/userdata.img
```

Example `package-file` for boot storage:

```
# NAME          Relative path
package-file    package-file
bootloader      Image-nor/MiniLoaderAll.bin
parameter       Image-nor/parameter.txt
uboot           Image-nor/uboot.img
```

Recommended packaging process:

1. Use `afptool` to package the main storage and boot storage `package-file` into two sub-images.
2. Use `rkImageMaker` to write the corresponding storage type for each sub-image. For example, main storage uses `-storage:emmc`, and boot storage uses `-storage:spinor`. Use the actual storage type of the platform.
3. Use `rkImageMaker -merge` to merge the two sub-images into the final `update.img`.

Example:

```
./afptool -pack ./ Image/update.img package-file-main
./rkImageMaker -RK3588 Image/MiniLoaderAll.bin Image/update.img main_update.img -
os_type:androidos -storage:emmc

./afptool -pack ./ Image-nor/update.img package-file-nor
./rkImageMaker -RK3588 Image-nor/MiniLoaderAll.bin Image-nor/update.img nor_update.img -
os_type:androidos -storage:spinor

./rkImageMaker -merge ./update.img ./main_update.img ./nor_update.img
```

> **Note:** The commands above describe the dual-storage packaging process. The actual vendor ID, storage type, loader file name, and script path depend on the SDK packaging scripts. For a complete script example, refer to the "Dual-Storage Firmware Packaging" section in `Rockchip_Developer_Guide_Dual_Storage_EN.pdf`.

When using updateEngine to upgrade dual-storage firmware, `--upgrade` still uses the item names in the package.

```
updateEngine --image_url=/udisk/update.img \
    --misc=update \
    --savepath=/userdata/update.img \
    --upgrade=bootloader,uboot,boot,rootfs \
    --reboot &
```

Dual-storage OTA packages are not recommended to upgrade `parameter` and `bootloader` by default. If this is required, verify that the boot storage and main storage partition tables match the product layout and that the start addresses of partitions not included in the upgrade do not change.

5.6 Custom Partition Packaging

Custom partitions do not require code changes in updateEngine. If the device partition table and package item name are consistent, the partition can be upgraded with `--upgrade=<name>`.

```
# NAME          Relative path
package-file    package-file
factory         Image/factory.img
```

Device-side command:

```
updateEngine --image_url=/userdata/update_factory.img \
    --update \
    --upgrade=factory \
    --reboot
```

5.7 Packaging Notes

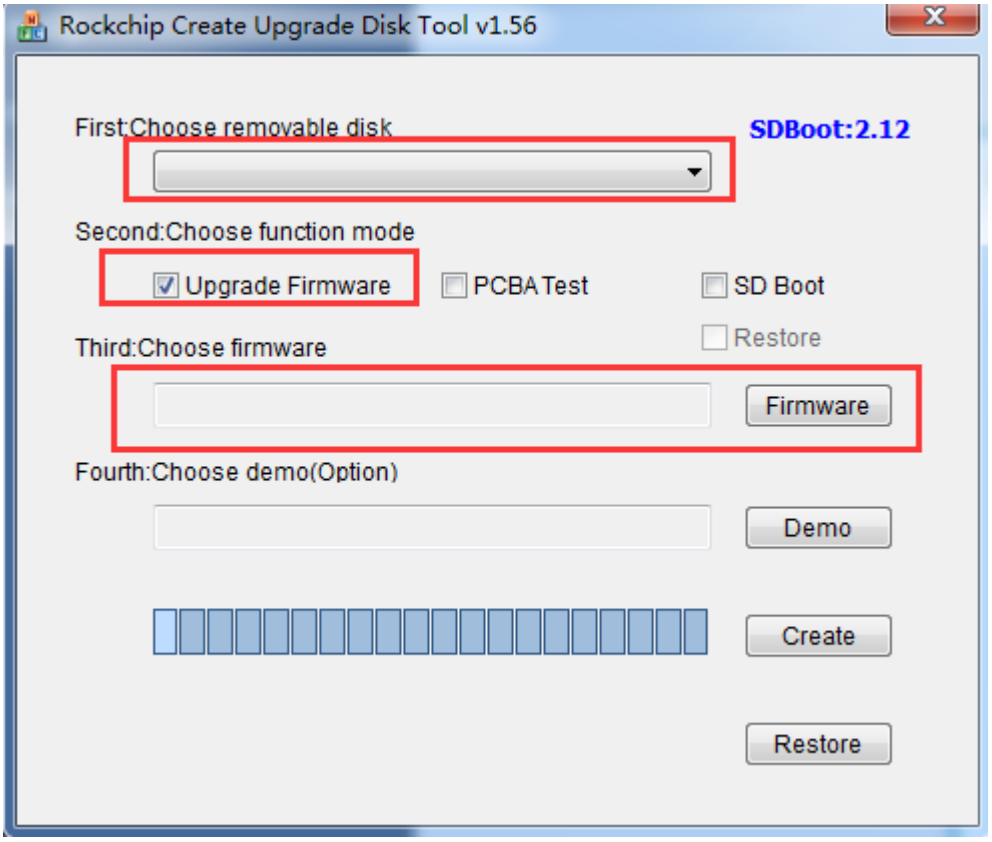
1. Names in `package-file` must match device-side partition names. Block devices depend on `/dev/block/by-name/<name>`, and MTD devices depend on partition names in `/proc/mtd`.
2. `--upgrade=all` does not include `parameter` and `bootloader`. Explicitly use `--upgrade=all,parameter` or `--upgrade=all,bootloader` when required.
3. When modifying the partition table, ensure that the start addresses of partitions not included in this upgrade do not change. If a partition start address changes, include the corresponding image in the package and add it to `--upgrade`.
4. Differential packages require the target device to match the old package baseline.
5. For network upgrades, specify `--savepath` to avoid downloading to a location with insufficient space or data loss after reboot.

6. SD/USB Boot Upgrade

SD card boot upgrade means using SDDiskTool to create an SD card upgrade boot disk and inserting it into the device for upgrade. USB boot upgrade uses the same updateEngine process with a different boot medium and `--update` parameter.

6.1 Create an SD Card Boot Disk

Use the SD card boot disk creation tool under `tools\windows\SDDiskTool`.



Select the packaged `update.img` in "Select Firmware".

After all preparations are complete, click the start button. A dialog is displayed when creation succeeds.

The SD card root directory contains two files. The selected upgrade firmware `update.img` is renamed to `sdupdate.img`.

Filesystem Format

SDDiskTool formats the SD card as FAT32 by default. FAT32 has the following limitations:

- A single file cannot exceed 4 GB.
- The SD card capacity is limited to 32 GB.

If the upgrade package exceeds 4 GB or the SD card capacity exceeds 32 GB, FAT32 formatting may fail. Modify `config.ini` under the SDDiskTool directory and change the filesystem type to exFAT or another filesystem that supports large files:

```
USER_DISK_FS=EXFAT
```

> **Note:** After changing the filesystem type, ensure that the target kernel enables support for the filesystem, for example `CONFIG_EXFAT_FS=y`. Otherwise the device cannot mount the SD card.

For the SD boot disk format, refer to `Rockchip_Developer_Guide_UBoot_Nextdev_EN.pdf`.

6.2 Instructions for Making SD Card Boot Disk in Recovery System Mode

For recovery system mode, use SDDiskTool to create `sdupdate.img` directly from `update.img`.

6.3 Instructions for Making SD Card Boot Disk in A/B System Mode

SD card boot upgrade depends on recovery mode boot. Because an A/B system partition table does not contain a recovery partition, it cannot directly boot into recovery mode from the SD card. To solve this, the SDK generates `update-sd.img` based on a non-A/B partition table that contains the recovery partition. This image is only used to create the SD card boot disk.

6.3.1 Prerequisites

1. `RK_SDCARD_UPDATE=y` is configured in defconfig.
2. `recovery.img` has been built, for example by `./build.sh recovery`.

Example configuration:

```
RK_PARAMETER="parameter-ab.txt"
RK_AB_UPDATE=y
RK_SDCARD_UPDATE=y
```

After the above conditions are met, run `./build.sh updateimg` to generate:

- **update-ab.img**: Packed with the A/B partition table and complete dual-slot system images. Used for device flashing.
- **update-sd.img**: Packed with a non-A/B partition table containing recovery. Used only to create the SD card boot disk.

> **Note:** If `RK_SDCARD_UPDATE` is not configured, `update-sd.img` is not generated.

6.3.2 Creation Steps

1. Use SDDiskTool to create an SD card boot disk from `update-sd.img`.
2. Copy `update-ab.img` to the SD card root directory and rename it to `sdupdate.img`.
3. Insert the SD card into the target device and power cycle the device. The device automatically enters recovery mode and performs the upgrade.

After creation, the SD card root directory is:

```
rksdfw.tag
sd_boot_config.config
sdupdate.img            # renamed from update-ab.img
```

> **Note:** The recovery partition in `update-sd.img` is only used to boot the SD card upgrade process. The firmware actually written to the target device is `sdupdate.img`, which uses the standard A/B partition table. After upgrade, the target device does not contain a recovery partition.

7. Restore Factory Settings

Writable configuration files are stored in the userdata partition. The factory firmware provides default configuration values. After the user modifies or generates configuration files, it may be necessary to clear these data and restore factory settings.

SDK implementation:

The RECOVERY + VOLUMEUP keys trigger factory reset. See:

buildroot/board/rockchip/rk3308/fs-overlay/etc/input-event-daemon.conf

buildroot/board/rockchip/rk3308/fs-overlay/usr/sbin/factory_reset_cfg

```
updateEngine --misc=wipe_userdata --reboot
```

Process:

1. Write the format command to misc partition offset 6 KB.
2. Reboot with `--reboot`.
3. `S21mountall.sh` detects the format command in misc.
4. Format userdata.

Optional parameter:

1. If `--reboot` is not specified, factory reset is performed after the next reboot.

8. Upgrade Program Details

8.1 Parameters

updateEngine supports partition upgrade, misc configuration, and factory reset. Command parameters:

```
*** updateEngine v2.0.0 ***
--misc=now                    Linux A/B mode: set the current partition bootable.
--misc=other                  Linux A/B mode: set the other partition bootable.
--misc=display                Display misc and A/B information.
--misc=update                 Recovery mode: set partitions to be upgraded.
--misc=wipe_userdata          Format data partition.
--misc_custom=read|write|clean Read, write, or clean custom misc command line.
--upgrade=<all|name,...>      Set partitions to be upgraded.
```

```
--reboot                        Reboot after the program completes.
--version_url=<url>             Path to the version file.
--check=<url>                   Same as version check input.
--image_url=<url>               Path to upgrade firmware.
--savepath=<path>               Save update.img to the specified path.
--version                       Display updateEngine version.
--help                          Display help information.
```

8.1.1 --misc

`now`: Used by Linux A/B mode to mark the current slot bootable.

Note: The `S99_bootcontrol` script runs this command at the final boot stage to mark the current partition bootable. Enable:

```
BR2_PACKAGE_RECOVERY_BOOTCONTROL=y
```

`other`: Used by Linux A/B mode to set the other slot as the upgraded slot. After reboot, the device tries to boot from the other slot.

Note: If updateEngine is used for upgrade, this is set automatically after the upgrade finishes and does not need to be set again.

`update`: Used by recovery mode. In the normal system, updateEngine first upgrades the recovery partition, then writes the remaining upgrade commands to misc/BCB. After rebooting into the recovery system, it continues upgrading the remaining partitions.

`display`: Used for debugging to display the misc partition data structure.

`wipe_userdata`: Write the factory reset command.

8.1.2 --update

Without parameters: mainly used by Linux A/B mode to upgrade the inactive slot directly in the current system.

8.1.3 --upgrade

Set the partitions to be upgraded.

Form	Description
`--upgrade=boot,recovery,rootfs`	Upgrade specified partitions.
`--upgrade=all`	Upgrade common partitions in the package except `package-file`, `parameter`, and `bootloader`.
`--upgrade=all,-userdata`	Upgrade partitions selected by `all` but exclude userdata.
`--upgrade=all,parameter`	Upgrade parameter/GPT in addition to all.
`--upgrade=all,bootloader`	Upgrade bootloader/IDBlock in addition to all.
`--upgrade=all,parameter,bootloader`	Commonly used for SD/USB boot full-firmware upgrade.

Description:

1. `--upgrade` is mandatory except for the SD/USB automatic upgrade flow.
2. `parameter` and `bootloader` are special upgrade items. They must be explicitly specified, and `all` does not include them by default.
3. `-name` excludes a partition from the upgrade scope, for example `--upgrade=all,-userdata`.
4. `!name` also excludes a partition, but an interactive shell may treat `!` as history expansion. Therefore, `-name` is recommended.
5. In A/B mode, `boot` can match `boot_a/boot_b`, and `system` can match `system_a/system_b`. updateEngine automatically selects the inactive slot.

8.1.4 --reboot

Reboot after updateEngine succeeds.

8.1.5 --version_url / --check

If a path is passed in, updateEngine compares it with the `RK_VERSION=` version value in `/etc/version` before upgrade.

Local path: read the version number from a local file.

Remote path: download the version file from a remote location. The remote version file format must match `/etc/version`.

8.1.6 --image_url

Specify the upgrade package path. The path can be a local path, an SD/USB path, or an HTTP/HTTPS URL when URL download support is enabled.

8.1.7 --savepath

Specify where to save the firmware. When the firmware path is a remote URL, the caller should explicitly specify a path such as `/userdata/update.img`.

8.1.8 parameter and bootloader Upgrade

> **Note:** Released products and mass-production OTA packages are not recommended to upgrade `parameter`/GPT and `bootloader`/IDBlock by default. These data are boot-critical. Power loss, write failure, or incorrect layout configuration may cause the device to fail to boot from the storage medium. Recovery through flashing tools or Maskrom mode may be required. Enable these items explicitly only when the partition table must be changed, IDBlock must be updated, or an SD/USB full-upgrade medium is being created.

`parameter` corresponds to GPT upgrade and is available only when `RK_GPT_UPDATE` is compiled. It must be explicitly specified:

```
updateEngine --image_url=/userdata/update.img --update --upgrade=all,parameter
```

When GPT is written, updateEngine generates standard GPT data from `parameter.txt` and writes two tables:

1. primary GPT: written at the beginning of the whole-disk target device.
2. backup GPT: written at the end of the whole-disk target device and aligned with the end of usable device capacity.

The backup GPT position depends on the logical block size and usable capacity of the target medium. Block devices use the whole-disk node. For NAND/NOR MTD devices, a master mtd node representing the whole flash, such as `/dev/mtd0`, must be exposed. Partition mtd nodes alone are not sufficient. If mtdparts split the device and the master node is not visible, check whether `CONFIG_MTD_PARTITIONED_MASTER=y` is enabled. Without a whole-chip mtd node, GPT, IDBlock, or full-flash writes cannot be reliably positioned and the upgrade is aborted.

`bootloader` corresponds to IDBlock upgrade and is available only when `RK_IDBLOCK_UPDATE` is compiled. It must be explicitly specified:

```
updateEngine --image_url=/userdata/update.img --update --upgrade=all,bootloader
```

When IDBlock is written, updateEngine generates boot data according to the medium type and Rockchip bootloader format, then writes it to a fixed boot area according to medium geometry. To improve boot reliability, multiple redundant IDBlock copies are written. The start offset, alignment, and copy count are determined by the storage backend. For example, SPI NOR places IDBlock in a fixed offset area, while NAND follows eraseblock size and bad-block handling rules.

MTD IDBlock upgrade also requires a whole NAND/NOR mtd node such as `/dev/mtd0`. If only partition nodes in `/proc/mtd` are available, updateEngine cannot write IDBlock to the boot area by absolute address and will not fall back to normal partition writing.

If GPT or IDBlock upgrade support is disabled at compile time, the corresponding item is skipped and will not fall back to normal partition writing. For low-level write methods and restrictions of different storage media, see “updateEngine Storage Write Implementation” in the appendix.

8.1.9 Pre-Write Checks

Before writing flash, updateEngine performs checks. If any check fails, the upgrade is aborted before writing.

Check Item	Description
Mount check	Common target partitions must not be mounted.
Capacity check	The write range must not exceed target device capacity.
GPT layout check	When upgrading parameter/GPT, if a partition start address changes, that partition must also be included in the upgrade list.
GPT mounted partition check	If a partition is mounted, its start address must not change in the new GPT, and its size must not shrink.

For example, when the upgrade package is stored in `/userdata/update.img`, do not upgrade userdata in the same operation. Otherwise the following error may occur:

```
precheck: userdata target /dev/block/by-name/userdata is mounted ... abort
```

8.2 Custom Partition Upgrade

The current version does not maintain a fixed partition table in code. If the package contains the corresponding item and the device has the same partition name, upgrade it with `--upgrade=<name>`.

Example: upgrade a custom partition named factory.

1. Ensure the device partition table contains factory.

Block devices usually require:

```
/dev/block/by-name/factory
```

MTD devices usually require a partition with the same name in `/proc/mtd`, for example:

```
mtd5: 00400000 00020000 "factory"
```

2. Add factory to `package-file`:

```
# NAME      Relative path
package-file package-file
factory      Image/factory.img
```

3. Run updateEngine:

```
updateEngine --image_url=/userdata/update.img \
    --misc=update \
    --upgrade=factory \
    --reboot
```

Common custom partitions use the normal image write path. Only special items such as `parameter` and `bootloader` use GPT/IDBlock dedicated write paths. For storage-specific write methods, see "updateEngine Storage Write Implementation" in the appendix.

9. Appendix

9.1 Firmware Packaging Tools

See Firmware Package Creation. Common packaging tool paths:

Platform	Tool Path	Description
Windows	`tools\windows\AndroidTool\rockdev`	Modify `package-file` and run `mkupdate.bat`.
Linux	`tools/linux/Linux_Pack_Firmware/rockdev`	Modify `package-file` and run `mkupdate.sh`.
Differential package	`tools/linux/Linux_Diff_Firmware/`	Run `mk-diff-ota.sh` to generate a differential OTA package.

9.2 updateEngine Storage Write Implementation

updateEngine selects the write method according to the storage medium of the target partition. Normal OTA upgrades usually do not require users to understand the low-level details. However, when upgrading NAND/NOR, UBI, parameter/GPT, bootloader/IDBlock, misc, or A/B metadata, the following restrictions should be understood.

Medium	Write Method	Description
eMMC/UFS/SATA/NVMe block devices	Write by offset through block device nodes	Common partitions use by-name partition nodes. Whole-disk data such as parameter/GPT uses the whole-disk node.
NAND	Use mtd-utils tools such as `flash_erase`, `nandwrite`, and `nanddump`	These tools handle bad blocks, OOB, and ECC.
NOR	Use MTD ioctl/read/write system calls	NOR does not have NAND-style bad-block skipping, but erase/write operations are still eraseblock based.
UBI	Write through UBI volume interfaces	UBI volume upgrades do not write raw MTD partitions directly.

When writing small data such as `misc`, A/B metadata, or BCB to NAND/NOR, the write range is usually smaller than one eraseblock. To preserve unchanged data in the same eraseblock, updateEngine performs read-modify-write for small writes that are not eraseblock-aligned.

Upgrades such as `parameter`/GPT, `bootloader`/IDBlock, and full-flash writes require absolute-position writes to the whole disk or whole flash. Block devices require the whole-disk node. NAND/NOR MTD devices must expose a master mtd node representing the whole flash, such as `/dev/mtd0`. If the system exposes only partition mtd nodes, updateEngine cannot reliably locate absolute GPT, IDBlock, or full-flash write positions. In that case, the upgrade is aborted and will not fall back to normal partition writing.

9.3 Misc Partition

The misc partition has no filesystem and stores boot configuration parameters. See `external/recovery/core/bootmsg.c`, `external/recovery/core/slot.c`, and related header files.

Offset	Purpose
2 KB	Linux A/B partition boot information
6 KB	Format command
16 KB	Communication between recovery system and normal system